

Nom:

Prénom:

Note:

Exercice 1 ►

/2 points

Sans aucune justification, donner le nombre d'états possibles codables avec:

- 1) 1 bit
- 2) 3 bits
- 3) 8 bits
- 4) n bits

Exercice 2 ►

/1 point

Convertir le nombre binaire suivant en décimal. Vous justifierez vos réponses.

1010 0011

Exercice 3 ►

/1 point

Convertir le nombre décimal suivant en binaire. Vous justifierez vos réponses.

159

Exercice 4 ►

/1 point

Convertir le nombre hexadécimal suivant en décimal. Vous justifierez vos réponses.

AF

Exercice 5 ►

/1 point

Convertir le nombre binaire suivant en hexadécimal. Vous justifierez vos réponses.

1110 0110

Exercice 6 ►

/1 point

Effectuer l'addition binaire suivante:

10110111+1101110

Exercice 7 ►

/2 points

Expliquer simplement comment déterminer si un nombre binaire est pair ou impair. Justifier.

Exercice 8 ►

/1 point

Ecrire une fonction `est_pair(n)` qui prend en paramètre un nombre écrit en binaire (au format chaîne de caractères) et renvoie `True` si le nombre est pair et `False` sinon.

```
def est_pair(n):  
    ''' Renvoie True si n est pair et False sinon  
    n chaîne de caractères représentant un nombre binaire  
    sortie: bool '''
```

```
assert est_pair('1010') == True    # 10 en décimal  
assert est_pair('1111') == False   # 7 en décimal
```

Exercice 9 ►

/2 points

Ecrire une fonction `nb_bits(n)` qui prend en paramètre un entier naturel `n` et renvoie le nombre de bits nécessaires pour l'écrire en binaire. Par exemple, `nb_bits(5)` renvoie 3 car $5 = 101_2$.
On pourra par exemple compter le nombre de divisions entières par 2 nécessaires pour obtenir 0.

```
def nb_bits(n):  
    ''' Renvoie le nombre de bits nécessaires pour écrire n en binaire  
    n entier naturel (int)  
    sortie: int '''
```

```
assert nb_bits(5) == 3    # 5 = 101_2  
assert nb_bits(10) == 4  # 10 = 1010_2  
assert nb_bits(16) == 5  # 16 = 10000_2
```

Exercice 10 ►

/2 points

Ecrire une fonction `nb_uns(val_bin)` qui prend en paramètre une chaîne de caractères `val_bin` représentant un nombre binaire et qui renvoie le nombre de bits à 1 dans cette chaîne.

```
def nb_uns(val_bin):  
    ''' Renvoie le nombre de bits à 1 dans val_bin  
        val_bin chaîne de caractères représentant un nombre binaire  
  
    assert nb_uns('101010') == 3  
    assert nb_uns('1111') == 4  
    assert nb_uns('0000') == 0
```

Exercice 11 ►

/1 point

Ecrire une fonction `multiplication(val_bin, puissance)` qui va multiplier un nombre binaire `val_bin` (sous forme de chaîne de caractères) par $2^{\text{puissance}}$ où `puissance` est un entier naturel.

```
def multiplication(val_bin, puissance):  
    ''' Multiplie le nombre binaire val_bin par 2^puissance  
        val_bin chaîne de caractères représentant un nombre binaire  
        puissance entier naturel (int)  
        sortie: str '''  
  
    assert multiplication('101', 3) == '101000'    # 5 * 2^3 = 40  
    assert multiplication('111', 2) == '11100'      # 7 * 2^2 = 28  
    assert multiplication('1001', 0) == '1001'      # 9 * 2^0 = 9
```

Exercice 12 ►

On dispose de deux chaînes binaires A et B de même longueur (8 bits).

On veut produire une chaîne binaire représentant leur somme.

Ecrire une fonction `addition_binaire(A, B)` qui prend en paramètres les deux chaînes binaires A et B et renvoie la chaîne binaire représentant leur somme. On ne pourra pas convertir les valeurs en décimal avant de faire l'addition.

```
def addition_binaire(A, B):  
    ''' Renvoie la chaîne binaire représentant la somme de A et B  
    A et B chaînes de caractères représentant des nombres binaires de même longueur (8 bits)  
    sortie: str '''
```

```
assert addition_binaire('00011010', '00000101') == '00011111'      # 26 + 5 = 31  
assert addition_binaire('11111111', '00000001') == '100000000'    # 255 + 1 = 256  
assert addition_binaire('10101010', '01010101') == '11111111'     # 170 + 85 = 255
```